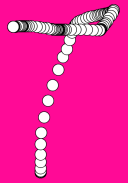
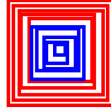


Basic Paint



```
function setup() {  
  createCanvas(windowWidth, windowHeight)  
  background (255,10,150)  
  frameRate(100)  
}  
  
function draw() {  
  ellipse(mouseX, mouseY,20)  
}
```

Molnar – One Square



```
let dimX = 400
let dimY = dimX
let num = 10
let reduction = dimX / num
let posX = 0
let posY = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(12, 23, 134)
  // rectMode describes where a rectangle is drawn from – usually from
  top left corner
  rectMode(CENTER)
  angleMode(DEGREES)

  // width and height are automatically from the canvas
  // and we defined it as WindowWidth and WindowHeight
  posX = width/2
  posY = height/2
}

function draw() {
  background(255)
  noFill()
  strokeWeight(5)
  stroke (0)

  //animation
  // calculte the dimension of the square based on sin fuction
  dimX = 200 + sin(frameCount * 4) * 50    // = 50+1
```

```

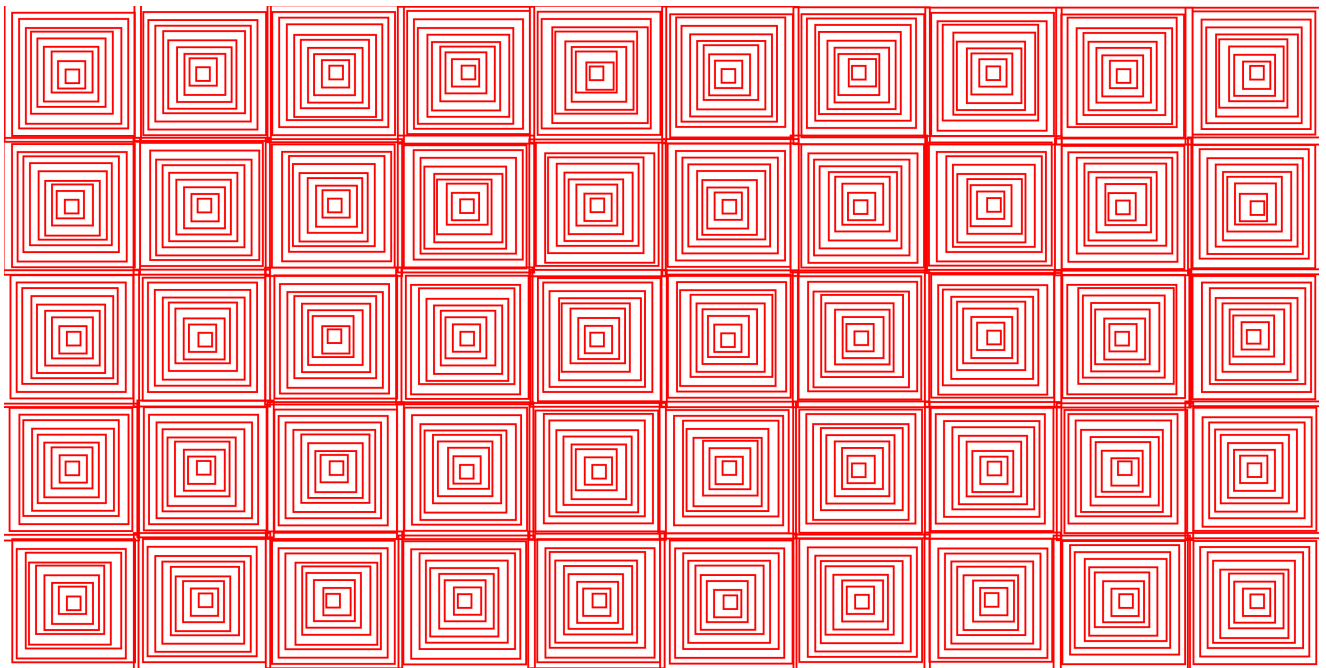
    num = 10
    reduction = dimX / num
// for loop
for (let i = 0; i < num; i++){
    let offsetX = random(7)
    let offsetY = random(7)

    stroke (255,0,0)
    if (i >= 5) {
        stroke (0, 0, 255)
    }

    square (
        posX + offsetX,
        posY + offsetY,
        (dimX) - (reduction * i)
    )}
}

```

Molnar – Full Screen



```

let dimX = 453
let dimY = dimX
let num = 17
let reduction = dimX / num
let posX = 0
let posY = 0

function setup() {
    createCanvas(windowWidth, windowHeight)
}

```

```

background(255)
rectMode(CENTER)
angleMode(DEGREES)
posX = width / 2
posY = height / 2
}

function draw() {
  background(255)
  noFill()
  strokeWeight(3)
  stroke(0)

  let numX = 10
  for(let i = 0; i < numX; i++){
    let dimension = width / numX
    let posX = dimension / 2 + (i * dimension)
    for (let j = 0; j < 5; j++){
      let posY = dimension / 2 + (j * dimension)
      tmcs(posX, posY, 4, dimension, 10)
    }

  }

  // animation!
  // calculate the dimension of the square
  // based on a sin function

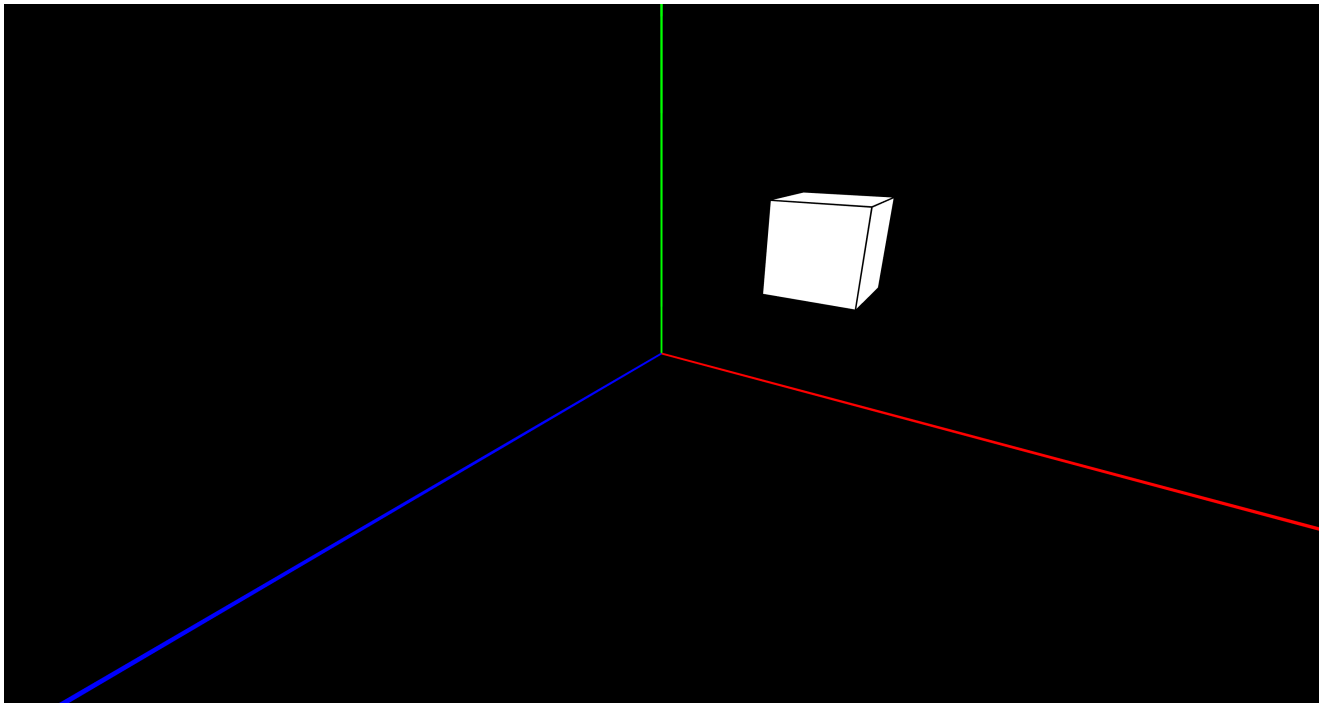
}
// this function draws squares at position
// x and y, and they move randomly
// a tiny bit
function tmcs(x, y, speed, dim, num){

  let dimension = dim + sin(frameCount * speed) * 10
  // num = 10
  let reduction = dimension / num

  for(let i = 0; i < num; i++) {
    let offsetX = random(7)
    let offsetY = random(7)
    stroke(255, 0, 0)
    strokeWeight(3)
    square(
      x + offsetX,
      y + offsetY,
      (dimension) - (reduction * i)
    )
  }
}

```

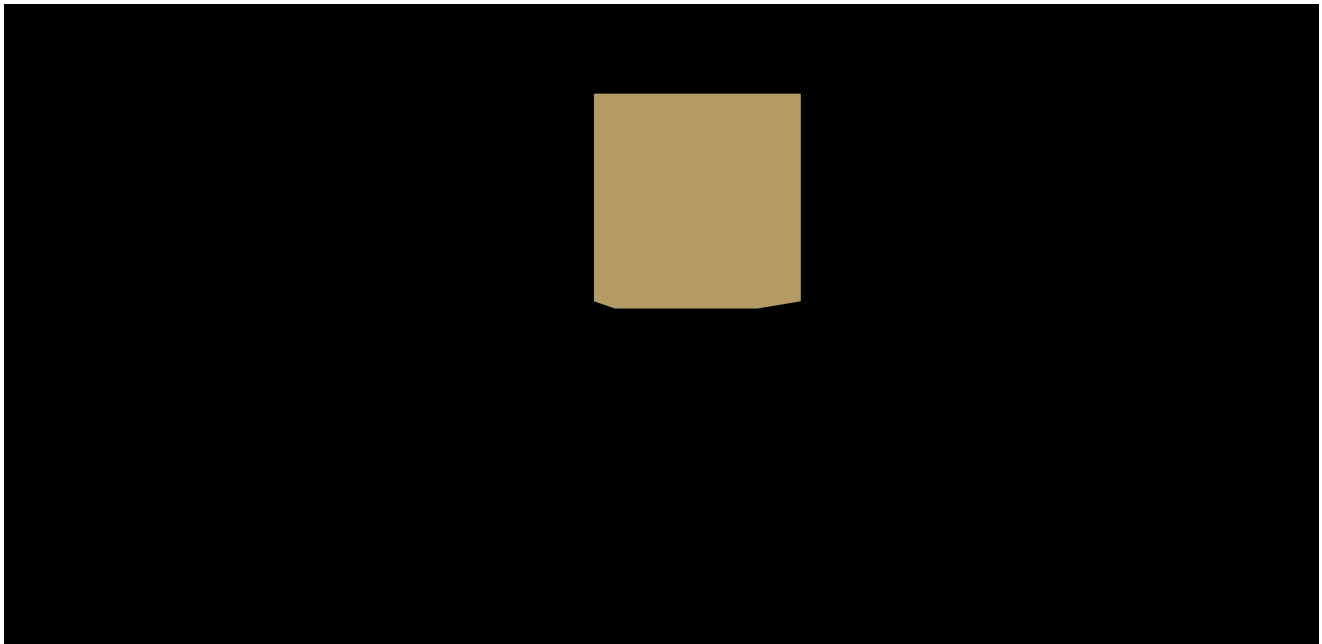
3D XYZ Lines



I dont understand what makes the cube move here.

```
function setup() {  
  createCanvas(windowWidth, windowHeight, WEBGL)  
}  
  
function draw() {  
  background(0)  
  orbitControl()  
  fill(255)  
  
  push()  
  translate(100, -100, -500)  
  box(200)  
  pop()  
  
  strokeWeight(3)  
  
  push()  
  stroke (255,0,0)  
  line(0,0,0, width,0,0)  
  stroke (0,255,0)  
  line(0,0,0, 0,-height,0)  
  stroke(0,0,255)  
  line(0,0,0, 0,0,1000)  
  pop()  
}
```

3D DVD Box



```
let x;
let y;
let z;
let xspeed;
let yspeed;
let zspeed;
let depth = 2000;
let dvd;
let r, g, b;

function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL);
  x = random(width);
  y = random(height);
  z = random(-depth, 0);
  xspeed = 4;
  yspeed = 4;
  zspeed = 4;
  pickcolor();
}

function pickcolor() {
  r = random(255);
  g = random(255);
  b = random(255);
}

function draw() {
  translate(-width/2, -height/2);
  background(0);
```

```
noStroke(255);
//fill(255);
fill(r, g, b);

push()
translate(x,y,z)
box(300)
//rect(x, y, 80, 60);
//image(dvd, x, y, 80, 60);
pop()

x = x + xspeed;
y = y + yspeed;
z = z + zspeed;

if (z >= 200) {
  zspeed = -zspeed;
  z = 200;
  pickcolor();
} else if (z <= -depth) {
  zspeed = -zspeed;
  z = -depth;
  pickcolor();
}

if (x >= width) {
  xspeed = -xspeed;
  x = width;
  pickcolor();
} else if (x <= 0) {
  xspeed = -xspeed;
  x = 0;
  pickcolor();
}

if (y >= height) {
  yspeed = -yspeed;
  y = height;
  pickcolor();
} else if (y <= 0) {
  yspeed = -yspeed;
  y = 0;
  pickcolor();
}
}
```